

6. Deadlock

Dr. Arun Kumar
Dept. of CSE, NIT Rourkela

Shareable vs Non-shareable resources

▶ ***Sharable*** resources

- ▶ can be used by multiple processes during their execution.
- ▶ e.g., CPU, I/O bus.

▶ ***Non-sharable*** resources

- ▶ cannot be used by multiple processes during their execution.
- ▶ e.g., printer

Static vs. Dynamic resource allocation

- ▶ **Static:** Allocate all the required resources *prior to the execution* of the process.
- ▶ **Dynamic:** Allocate only the initially required resources to start up the process; additional resources will be allocated *dynamically*.

Static	Dynamic
▶ <u>Simpler</u> to implement ▶ Process is <u>guaranteed</u> to complete in definite time ▶ Resource utilization less ▶ Deadlock will never occur	▶ Comparatively complex ▶ No certainty about the process completion ▶ <u>Better resource utilization</u> ▶ May lead to <u>deadlock</u>

Deadlock

- ▶ **Deadlock** is a *situation* where a *set of processes are blocked*.
- ▶ Each process in the set *waits for some event* from *some other process in the same set*.
- ▶ Example #1
 - ▶ A system has 2 disk drives
 - ▶ P_1 and P_2 each hold one disk drive and each needs the other one
- ▶ Example #2
 - ▶ Semaphores A and B , initialized to 1

P_0	P_1
wait(A);	wait(B)
wait(B);	wait(A)

Deadlock

- ▶ Example #3
 - ▶ Adharcard and PAN

System Model

Each process *utilizes a resource* as follows:

- **Request** – If the request cannot be granted immediately, the process must *wait* until the resource(s) required to become available. The system, for example, uses the functions `open()`, `malloc()`, `new()`, and `request ()`.
- **Use** – The process makes *use of the resource*, such as printing to a printer or reading from a file.
- **Release** – The process *relinquishes the resource*, allowing it to be used by other processes.

Deadlock Characterization – Necessary Conditions

Deadlock can arise if four conditions hold simultaneously.

- ▶ **Mutual exclusion:** only one process at a time can use a resource. *Non-sharable resource*
- ▶ **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.
- ▶ **No pre-emption:** a resource can be released only voluntarily by the process holding it after that process has completed its task
 - ▶ High priority processes or another process can't force any other process to release its resources.
 - ▶ All processes have same priority, democracy.

Deadlock Characterization – Necessary Conditions

- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0

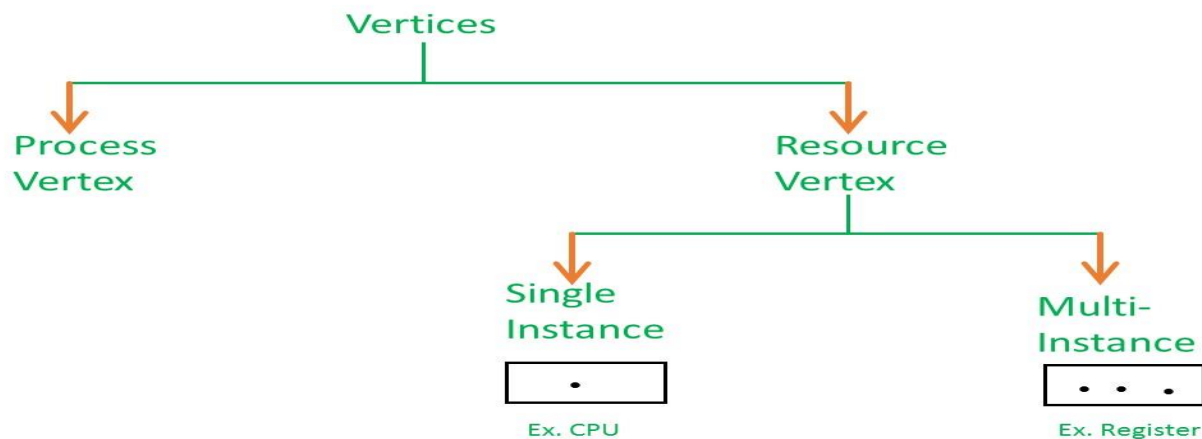
Resource-Allocation Graph

- ▶ **Resource Allocation Graph (RAG)** is to explain the state of the system in terms of **processes and resources**.
- ▶ How many resources are available, how many are allocated and what is the request of each process. Everything can be *represented* in terms of the diagram.
- ▶ We know that any graph contains **vertices (V)** and **edges (E)**. So *RAG* also contains vertices and edges. In RAG vertices are two type –
 - ▶ **Process vertex** – Every **process** will be represented as a process vertex. Generally, the process will be represented with a **circle**.
 - ▶ **Resource vertex** – Every **resource** will be represented as a resource vertex. Generally, the resources are represented with a **rectangular**.

Resource-Allocation Graph

Resource vertex are of two types

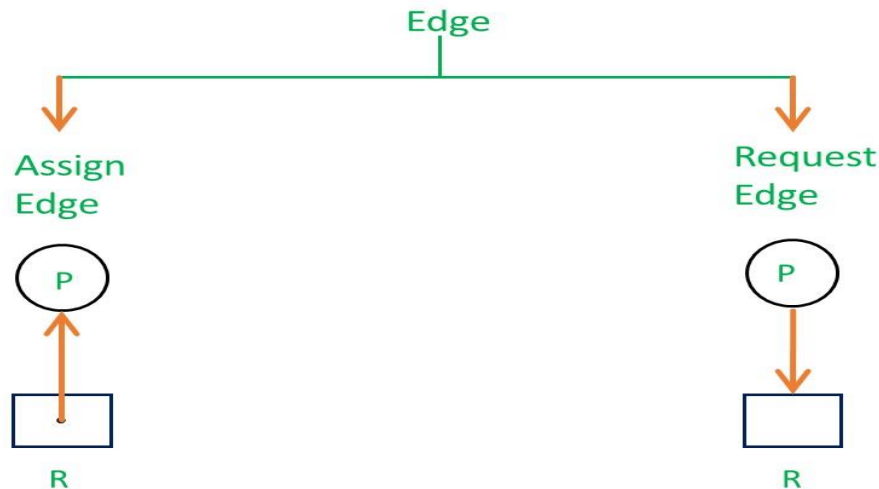
- ▶ ***Single instance type resource*** – It represents as a box, inside the box, there will be ***one dot***.
- ▶ ***Multi-resource instance type resource*** – It also represents as a box, inside the box, there will be ***many dots present***.
- ▶ So the *number of dots* indicate *how many instances are present* of each resource type.



Resource-Allocation Graph

- Now coming to the edges of RAG. There are ***two types of edges*** in RAG –

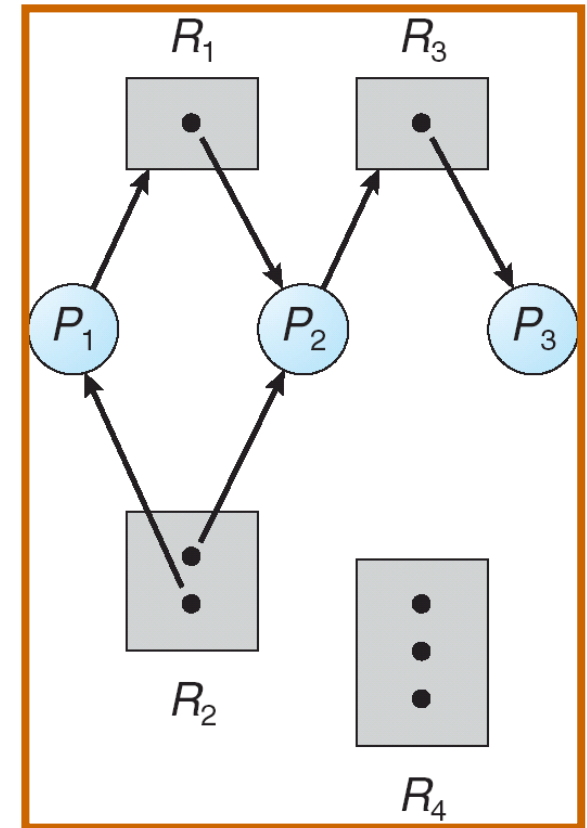
- **Assign Edge** – If you already *assign a resource to a process* then it is called Assign edge.
- **Request Edge** – It means in future the process *might want some resource* to complete the execution, that is called request edge.



Resource-Allocation Graph

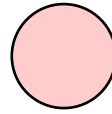
A set of vertices V and a set of edges E .

- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **Request edge** – directed edge $P_i \rightarrow R_j$
- **Assignment edge** – directed edge $R_j \rightarrow P_i$



Resource-Allocation Graph (Cont.)

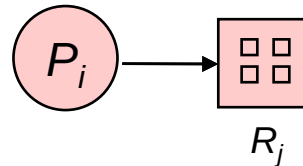
- Process



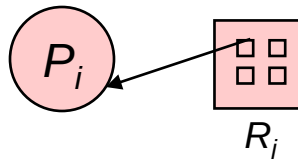
- Resource Type with 4 instances



- P_i requests instance of R_j

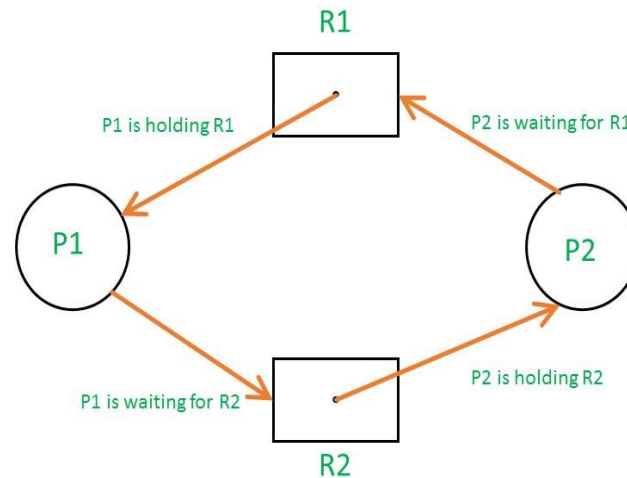


- P_i is holding an instance of R_j



Example 1 (Single instances RAG)

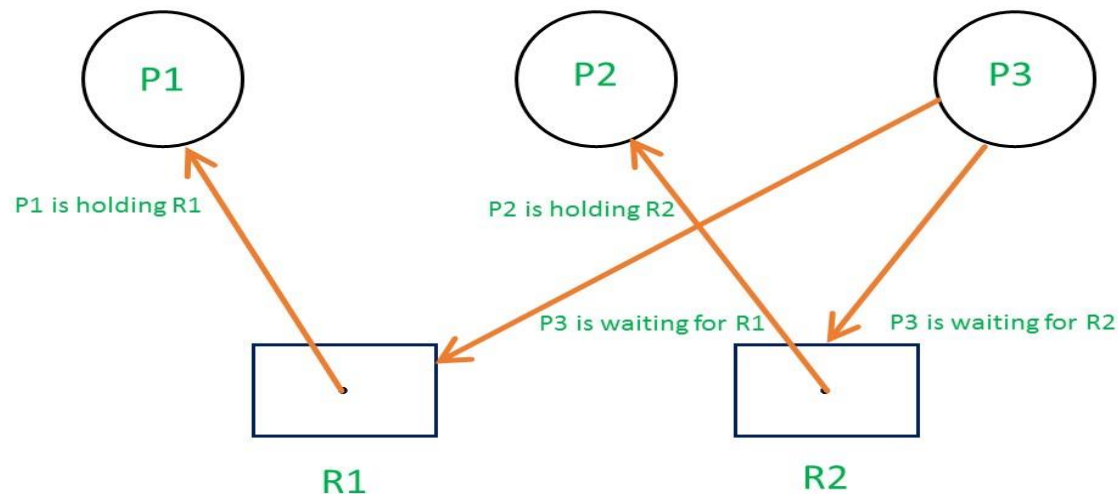
- If there is *a cycle* in the Resource Allocation Graph and each resource in the cycle provides only *one instance*, then the processes will be in **deadlock**.
- Example, if process P1 holds resource R1, process P2 holds resource R2 and process P1 is waiting for R2 and process P2 is waiting for R1, then process P1 and process P2 will be in deadlock.



SINGLE INSTANCE RESOURCE TYPE WITH DEADLOCK

Example 1 (Single instances RAG)

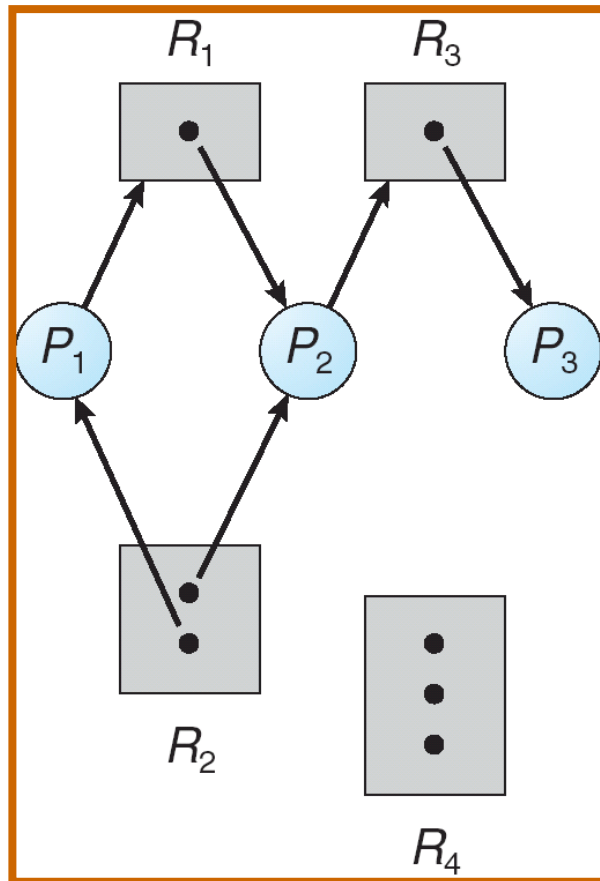
- Processes P1 and P2 acquiring resources R1 and R2 while process P3 is waiting to acquire both resources. In this example, there is ***no deadlock*** because there is ***no circular dependency***. So ***cycle in single-instance resource type*** is the sufficient condition for deadlock.



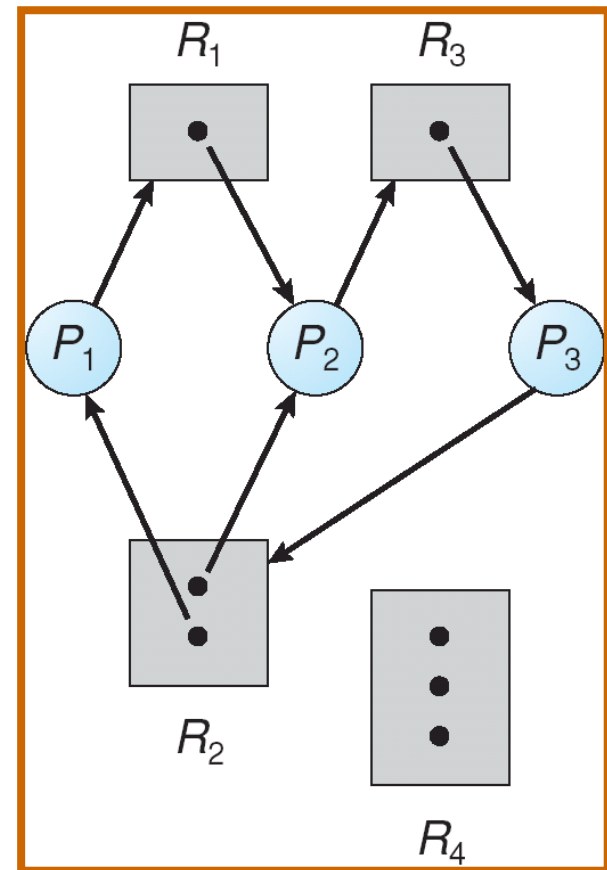
SINGLE INSTANCE RESOURCE TYPE WITHOUT DEADLOCK

Example 2 (Multi-instances RAG) - Resource Allocation Graph With A Deadlock

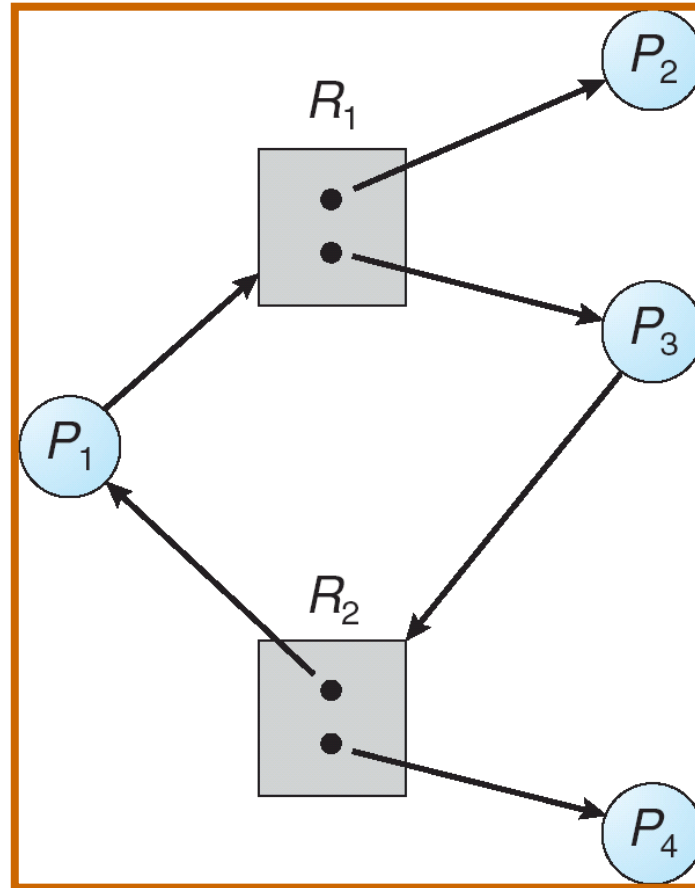
Before P_3 requested an instance of R_2



After P_3 requested an instance of R_2



Graph With A Cycle But No Deadlock



Process P_4 may *release* its instance of resource type R_2 . That resource can then be *allocated* to P_3 , thereby ***breaking the cycle***.

Relationship of cycles to deadlocks

- If a resource allocation graph contains **no cycles** $\Rightarrow$ ***no deadlock***
- If a resource allocation graph contains ***a cycle*** and if **only one instance** exists per resource type $\Rightarrow$ ***deadlock***
- If a resource allocation graph contains ***a cycle*** and and if **several instances** exists per resource type $\Rightarrow$ ***possibility of deadlock***

Methods for Handling Deadlocks

▶ ***Prevention***

- ▶ Ensure that the system will never enter a deadlock state

▶ ***Avoidance***

- ▶ Ensure that the system will never enter an unsafe state

▶ ***Detection***

- ▶ Allow the system to enter a deadlock state and then recover

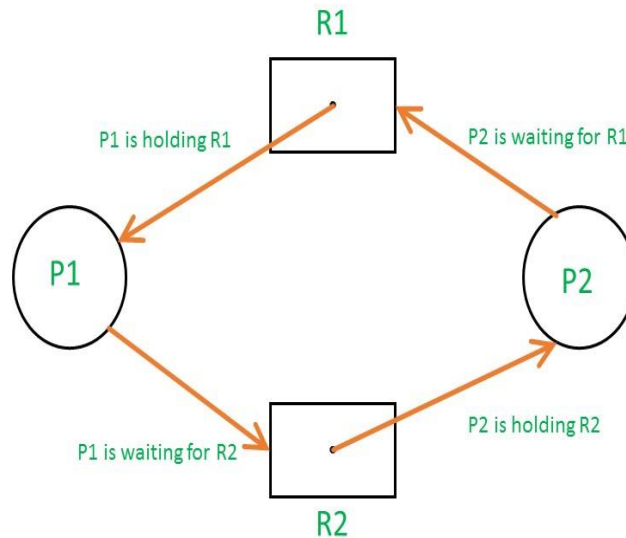
▶ ***Do Nothing/ Ignorance/Ostrich Approach***

- ▶ Ignore the problem and let the user or system administrator respond to the problem; used by most operating systems, including Windows and UNIX

Deadlock Prevention

Restrain at least one of the causes of deadlock

- **Mutual Exclusion** – The mutual-exclusion condition must hold for non-sharable resources.
- You can *increase the resources* (printer, CPU) but that is not a solution.



Deadlock Prevention

► Hold and Wait – Three methods:

1. **Conservative Approach:** Process is allowed to start execution if and only if it has *acquired all the resources*.
 - Less efficient, not implementable, easy.
2. **Do not hold:** Process will acquire desired resources, before making *any fresh request* it must *release all the resources it currently holds*.
3. **Wait timeout:** A *maximum time up* to which a process can wait. After which a process must *release all the holding resources*.

Deadlock Prevention

► No Preemption

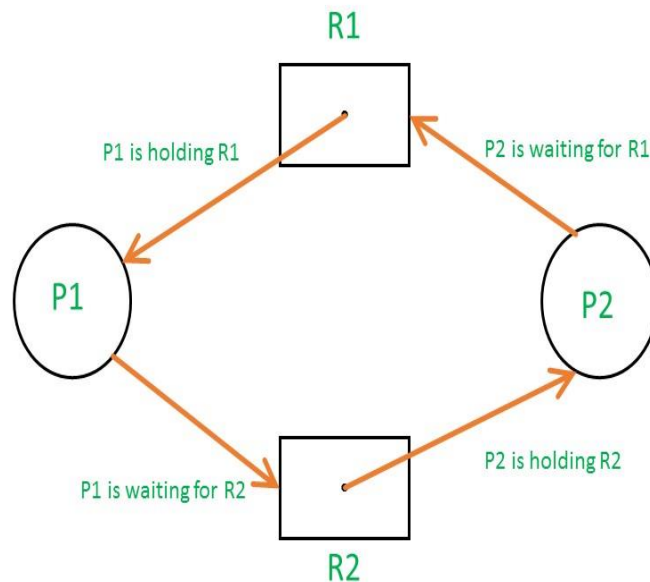
- If a process that is holding some resources *requests another resource* that cannot be immediately allocated to it, then all *resources currently being held* are ***preempted.***
- *Preempted resources* are added to the *list of available resources.*
- A process will be restarted only when it can *regain its old resources*, and the *new ones that it is requesting.*

► ***High priority - Starvation***

Deadlock Prevention

► Circular Wait:

- **Order** of request is important when acquiring the Resources.



Deadlock Prevention

► Circular Wait

- Impose a total **ordering of** all resource types, and require that each process requests resources in an increasing order of enumeration. For example:

1. R1
2. R2
3. R3
4. R4

If P_i holds R3 and requests for R2 then it has to release R3 first and re-request for R2 then R3.

Deadlock Avoidance and Safe State

- ▶ When a process requests an available resource, the system must decide if immediate allocation leaves the system in a ***safe state***
- ▶ A system is in a safe state only if there exists a ***safe sequence of execution***
- ▶ Total number of resources=12

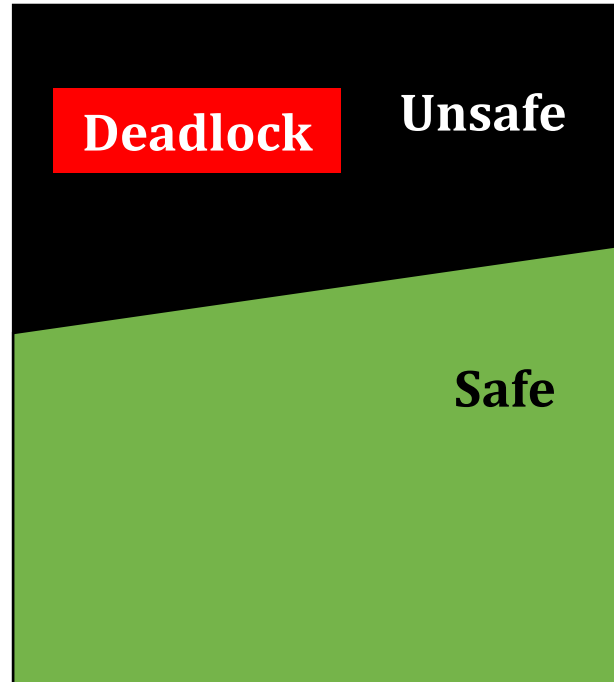
Process	Max Needs	Allocated	Current Needs
P0	10	5	5
P1	4	2	2
P2	7	3	4

- ▶ P0 requests one more resource dynamically. Can it be granted?
 - ▶ $\langle P1, P2, P0 \rangle$ is a safe sequence of execution

Safe State (continued)

- ▶ If a system is in safe state $\Rightarrow$ **no deadlocks**.
- ▶ If a system is in unsafe state $\Rightarrow$ ***possibility of deadlock***.
- ▶ ***Avoidance*** $\Rightarrow$ ensure that a system will never enter an unsafe state.

Safe, Unsafe, Deadlock State



Avoidance algorithms

- ▶ For a single instance of a resource type, use a *resource-allocation graph (RAG)*
- ▶ For multiple instances of a resource type, use the *banker's algorithm*

Banker's Algorithm

- ▶ When a process makes a request for a set of resources,
- ▶ Resource Request Algorithm:
 - ▶ Assume that the request is granted,
 - ▶ Update the system state accordingly,
 - ▶ Then, determine if the result is a safe state (Safety Algo)
 - If so, grant the request
 - if not, block the process until it is safe to grant the request.

Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- ▶ **Available:** Vector of length m . If **Available** $[j] = k$, there are k instances of resource type R_j available.
- ▶ **Max:** $n \times m$ matrix. If **Max** $[i,j] = k$, then process P_i may request at most k instances of resource type R_j .
- ▶ **Allocation:** $n \times m$ matrix. If **Allocation** $[i,j] = k$ then P_i is currently allocated k instances of R_j .
- ▶ **Need:** $n \times m$ matrix. If **Need** $[i,j] = k$, then P_i may need k more instances of R_j to complete its task.

$$\text{Need } [i,j] = \text{Max}[i,j] - \text{Allocation } [i,j]$$

Safety Algorithm

- ▶ 1) Let Work and Finish be vectors of length 'm' and 'n' respectively.
Initialize: Work = Available
Finish[i] = false; for i=1, 2, 3, 4....n
- ▶ 2) Find an i such that both
 - a) Finish[i] = false
 - b) $\text{Need}_i \leq \text{Work}$if no such i exists goto step (4)
- ▶ 3) $\text{Work} = \text{Work} + \text{Allocation}[i]$
Finish[i] = true
goto step (2)
- ▶ 4) if Finish [i] = true for all i
then the system is in a safe state

Resource-Request Algorithm (The Bankers Algorithm)

- ▶ 1) If $\text{Request}_i \leq \text{Need}_i$
Goto step (2) ; otherwise, raise an error condition, since the process has exceeded its maximum claim.
- ▶ 2) If $\text{Request}_i \leq \text{Available}$
Goto step (3); otherwise, P_i must wait, since the resources are not available.
- ▶ 3) Have the system pretend to have allocated the requested resources to process P_i by modifying the state as follows:
 $\text{Available} = \text{Available} - \text{Request}_i$
 $\text{Allocation}_i = \text{Allocation}_i + \text{Request}_i$
 $\text{Need}_i = \text{Need}_i - \text{Request}_i$

Banker's Algorithm: Example

Initially Available=(10,5,7)

Process	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	3	3	2
P1	2	0	0	1	2	2			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

P1 makes Request₁ = (1, 0, 2)

Banker's Algorithm: Example

Updated State.

Let's check for a safe sequence.

Process	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	2	3	0
P1	3	0	2	0	2	0			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

P1 makes $\text{Request}_1 = (1, 0, 2)$

Banker's Algorithm: Example

P1 can be completed.

Process	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	5	3	2
P1	0	0	0	0	0	0			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

P1 makes $\text{Request}_1 = (1, 0, 2)$

Banker's Algorithm: Example

P3 can be completed.

Process	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	7	4	3
P1	0	0	0	0	0	0			
P2	3	0	2	6	0	0			
P3	0	0	0	0	0	0			
P4	0	0	2	4	3	1			

P1 makes $\text{Request}_1 = (1, 0, 2)$

Banker's Algorithm: Example

P4 can be completed.

Process	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	7	4	5
P1	0	0	0	0	0	0			
P2	3	0	2	6	0	0			
P3	0	0	0	0	0	0			
P4	0	0	0	0	0	0			

P1 makes $\text{Request}_1 = (1, 0, 2)$

Banker's Algorithm: Example

P0 can be completed.

Process	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	0	0	0	0	0	7	5	5
P1	0	0	0	0	0	0			
P2	3	0	2	6	0	0			
P3	0	0	0	0	0	0			
P4	0	0	0	0	0	0			

P1 makes $\text{Request}_1 = (1, 0, 2)$

Banker's Algorithm: Example

P2 can be completed.

Pro cess	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	0	0	0	0	0	10	5	7
P1	0	0	0	0	0	0			
P2	0	0	0	0	0	0			
P3	0	0	0	0	0	0			
P4	0	0	0	0	0	0			

P1 makes $\text{Request}_1 = (1, 0, 2)$

Banker's Algorithm: Example

Safe Sequence: <P1,P3,P4,P0,P2>

Process	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	2	3	0
P1	3	0	2	0	2	0			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

P1 makes $\text{Request}_1 = (1, 0, 2) \rightarrow \text{GRANTED}$

Banker's Algorithm: Example

Process	Allocation				Need				Available		
	A	B	C		A	B	C		A	B	C
P0	0	1	0		7	4	3		2	3	0
P1	3	0	2		0	2	0				
P2	3	0	2		6	0	0				
P3	2	1	1		0	1	1				
P4	0	0	2		4	3	1				

Now, P4 makes **Request₄ = (3, 3, 0)** --> **NOT AVAILABLE**

Banker's Algorithm: Example

Process	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	2	3	0
P1	3	0	2	0	2	0			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

Now, P0 makes **Request₀** = (0, 2, 0)

Banker's Algorithm: Example

Updated State.

Let's check for a safe sequence.

Pro cess	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	3	0	7	2	3	2	1	0
P1	3	0	2	0	2	0			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

Now, P0 makes $\text{Request}_0 = (0, 2, 0)$

Banker's Algorithm: Example

No process can complete

Process	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	3	0	7	2	3	2	1	0
P1	3	0	2	0	2	0			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

Now, P0 makes $\text{Request}_0 = (0, 2, 0) \rightarrow \text{NOT GRANTED}$

Banker's Algorithm: Example

Revert back to previous state

Process	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	2	3	0
P1	3	0	2	0	2	0			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

Disadvantage: Time complexity is very high.

Deadlock Detection and Recovery

- ▶ If deadlocks are not avoided, then another approach is to ***detect*** when they have occurred and ***recover*** somehow.
- ▶ In addition to the performance hit of constantly checking for deadlocks, a policy/algorithm must be in place for ***recovering from deadlocks***, and there is potential for lost work when processes must be aborted or have their resources pre-empted.

Deadlock Detection and Recovery

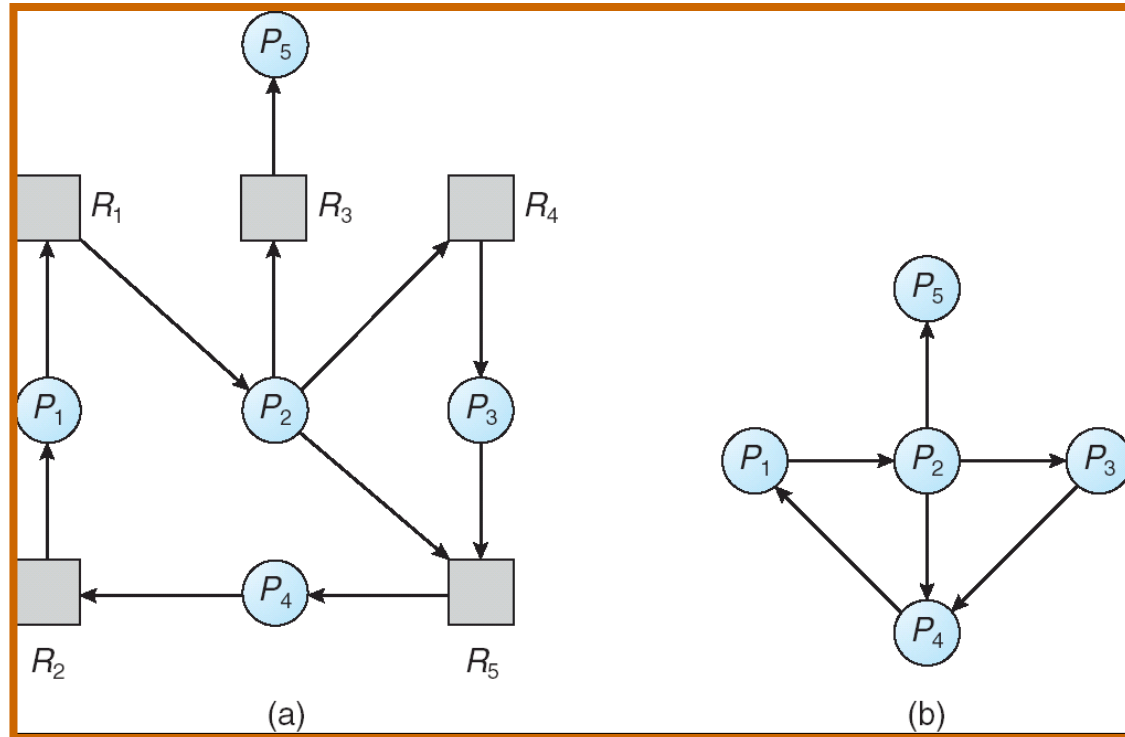
- ▶ An algorithm that *examines* the state of the system to detect whether a **deadlock** has occurred
- ▶ And an algorithm to *recover* from the deadlock
- ▶ Requires *overhead* that includes run-time cost of maintaining the necessary information and executing the *detection* algo.

Deadlock Detection:

Single Instance of Each Resource Type

- ▶ Requires the creation and maintenance of a wait for graph (WFG)
 - ▶ variant of the resource-allocation graph
 - ▶ The graph is obtained by **removing** the resource nodes from a RAG and **collapsing** the appropriate edges
 - ▶ Consequently; all nodes are processes
 - ▶ $P_i \rightarrow P_j$ if P_i is waiting for P_j
- ▶ Periodically invoke an algorithm that searches for a cycle in the graph
 - ▶ If there is a **cycle**, there exists a deadlock
 - ▶ An algorithm to detect a cycle in a graph requires $O(|V|+|E|)$ operations, where V is the set of vertices and E is the set of edges in the graph

Deadlock Detection: Single Instance of Each Resource Type



Resource-Allocation Graph

Corresponding wait-for graph

Deadlock Detection:

Multiple Instances of Resource Types

The algorithm employs several time varying data structures:

- ▶ **Available-** A vector of length m indicates the number of available resources of each type.
- ▶ **Allocation-** An $n \times m$ matrix defines the number of resources of each type currently allocated to a process. Column represents resource and resource represent process.
- ▶ **Request-** An $n \times m$ matrix indicates the current request of each process. If $\text{request}[i][j]$ equals k then process P_i is requesting k more instances of resource type R_j .

Deadlock Detection: Multiple Instances of Resource Types

1. Let *Work* and *Finish* be vectors of length *m* and *n* respectively. Initialize *Work* = *Available*. For $i=0, 1, \dots, n-1$, if $Request_i = 0$, then $Finish[i] = \text{true}$; otherwise, $Finish[i] = \text{false}$.
2. Find an index *i* such that both
 - a) $Finish[i] == \text{false}$
 - b) $Request_i \leq Work$If no such *i* exists go to step 4.
3. $Work = Work + Allocation_i$
 $Finish[i] = \text{true}$
Go to Step 2.
4. If $Finish[i] == \text{false}$ for some i , $0 \leq i < n$, then the system is in a deadlocked state. Moreover, if $Finish[i] == \text{false}$ the process P_i is deadlocked.

Deadlock Detection Algorithm: Example

Sequence $\langle P0, P2, P3, P1, P4 \rangle$ results in $Finish[i] == true$ for all i
System is not in deadlocked state

Pro cess	Allocation				Request				Available		
	A	B	C		A	B	C		A	B	C
P0	0	1	0		0	0	0		0	0	0
P1	2	0	0		2	0	2				
P2	3	0	3		0	0	0				
P3	2	1	1		1	0	0				
P4	0	0	2		0	0	2				

Deadlock Detection Algorithm: Example

Finish[0] is True but **P1, P2, P3, P4** are deadlocked.

Process	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	3	0	0	1			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

Detection-Algorithm Usage

- ▶ ***When should the deadlock detection be done? Frequently, or infrequently?***
- ▶ The answer may depend on how frequently deadlocks are expected to occur, as well as the possible consequences of not catching them immediately.

There are two obvious approaches, each with trade-offs:

- ▶ Do deadlock detection after every resource allocation which cannot be immediately granted. This has the advantage of detecting the deadlock right away, while the minimum number of processes are involved in the deadlock.
 - ▶ The drawback of this approach is the extensive overhead and performance hit caused by checking for deadlocks so frequently.
- ▶ Do deadlock detection only when there is some clue that a deadlock may have occurred, such as when CPU utilization reduces to 40% or some pre set threshold.
 - ▶ The advantage is that deadlock detection is done much less frequently, but the drawback is that it becomes impossible to detect the processes involved in the original deadlock, and so deadlock recovery can be more complicated and damaging to more processes.

Recovery from Deadlock

There are three basic approaches to recovery from deadlock:

- ▶ Inform the system operator, and allow him/her to take manual intervention.
- 1. Terminate one or more processes involved in the deadlock
- 2. Pre-empt resources.

Recovery from Deadlock: Process Termination

- ▶ Abort all deadlocked processes
 - ▶ This approach will break the deadlock, but at great expense
- ▶ **Abort** one process at a time until the deadlock cycle is eliminated
 - ▶ This approach incurs considerable overhead, since, after each process is aborted, a deadlock-detection algorithm must be re-invoked to determine whether any processes are still deadlocked
- ▶ Many factors may affect which process is chosen for termination
 - ▶ What is the **priority** of the process?
 - ▶ How long has the process run so far and how much longer will the process need to run before completing its task?
 - ▶ How many and what type of resources has the process used?
 - ▶ How many more resources does the process need in order to finish its task?
 - ▶ **How many** processes will need to be terminated?

Recovery from Deadlock: Resource Preemption

- ▶ With this approach, we successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken
- ▶ When preemption is required to deal with deadlocks, then three issues need to be addressed:
 - ▶ **Selecting a victim** – Which resources and which processes are to be preempted?
 - ▶ **Rollback** – If we preempt a resource from a process, what should be done with that process?
 - ▶ **Starvation** – How do we ensure that starvation will not occur? That is, how can we guarantee that resources will not always be preempted from the same process?

Summary

- ▶ Both deadlock avoidance and deadlock detection and recovery are expensive
- ▶ **Do nothing:** ignore the problem altogether and pretend that deadlocks never occur in the system (used by Windows and Unix)
 - ▶ System administrator/user will restart the processes